

COAL LAB #6: BUBBLE SORTING & INDIRECT ACCESSING:

NAME:ARSAM ALI

ROLL-NO:23P_0038

PROGRAM/SECTION:AI-4A

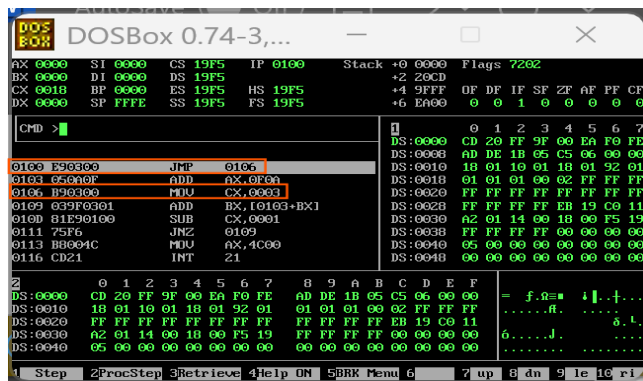
DATE:12-3-2025

Task 1: Explain relative addressing:

Code:

```
1  [org 0x0100]
2  jmp start
3
4
5  data: db 5, 10, 15
6
7  start:
8      mov cx, 3
9
10
11 l1:
12     add bx, [data+bx]
13     sub cx, 1
14     jnz l1
15
16     mov ax, 0x4c00
17     int 0x21
```

When the above code is executed we see the following on debugger:



Here, the jump taken is equal to bytes consumed by the box highlighted in code section

```

1      [org 0x0100]
2      jmp start
3
4
5      data: db 5, 10, 15, 20
6
7      start:
8          mov cx, 3
9
10     l1:
11         add bx, [data+bx]
12         sub cx, 1
13         jnz l1
14
15         mov ax, 0x4c00
16         int 0x21
17

```

Highlighted line has been changed.

Conclusion: indirect addressing first calculate how many bytes have been consumed by the label and takes the jump according to that.

Bubble sort: Explain the bubble sorting process and tell how do you know your data is sorted.

In Bubble sorting we need a flag to check if swapping is done or not so we make a flag of our own. (if the flag remains 0 in a phase we say data is sorted)

Note: The bsort jump should be place with the start acting as outer loop.

```
1  [org 0x0100]
2  jmp start
3
4  data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
5  swap: db 0
6
7  start:
8  → bsort:
9      mov bx, 0
10     mov byte [swap], 0
11
12     loop1:
13         mov ax, [data+bx]
14         cmp ax, [data+bx+2]
15         jbe noswap
16
17         mov dx, [data+bx+2]
18         mov [data+bx+2], ax
19         mov [data+bx], dx
20         mov byte [swap], 1
21
22     noswap:
23         add bx, 2
24         cmp bx, 18
25         jne loop1
26
27         cmp byte [swap], 1
28         je bsort
29
30     mov ax, 0x4c00
31     int 0x21
```

DOSBox 0.74-3,...

AX	0000	SI	0000	CS	19F5	IP	0100	Stack	+0 0000	Flags	7202
BX	0000	DI	0000	DS	19F5				+2 20C0		
CX	0051	BP	0000	ES	19F5	HS	19F5		+4 9FFF	OF	DF
DX	0000	SP	FFFF	SS	19F5	FS	19F5		+6 E000	0	0
										1	0
										2	0
										3	0
										4	0
										5	0
										6	0
										7	0
										8	0
										9	0
										10	0
										11	0
										12	0
										13	0
										14	0
										15	0
										16	0
										17	0
										18	0
										19	0
										20	0
										21	0
										22	0
										23	0
										24	0
										25	0
										26	0
										27	0
										28	0
										29	0
										30	0
										31	0

0100 E91500 JMP 0118

0103 3000 CMP AL,00

0105 37 AAA

0106 0020 ADD DI,1,CH

0109 0032 ADD [BP+SI],BH

010A 002B ADD [BX+SI],CH

010C 0023 ADD [BP+DI],AH

010E 0019 ADD [BX+DI],BL

DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.00= i.l.+...

DS:0010 1B 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FFA.

DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11A.

DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00J.

DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

As highlighted our swap flag is at location 0x0117 which is Initially 0.

DOSBox 0.74-3,...

AX	0000	SI	0000	CS	19F5	IP	0118	Stack	+0 0000	Flags	7200
BX	0000	DI	0000	DS	19F5				+2 20C0		
CX	0051	BP	0000	ES	19F5	HS	19F5		+4 9FFF	OF	DF
DX	0000	SP	FFFF	SS	19F5	FS	19F5		+6 E000	0	0
										1	0
										2	0
										3	0
										4	0
										5	0
										6	0
										7	0
										8	0
										9	0
										10	0
										11	0
										12	0
										13	0
										14	0
										15	0
										16	0
										17	0
										18	0
										19	0
										20	0
										21	0
										22	0
										23	0
										24	0
										25	0
										26	0
										27	0
										28	0
										29	0
										30	0
										31	0

0116 0000 AND [BX+SI],AL

0118 BB0000 MOV BX,0000

011B C000F0105 MOV [0117],00

0120 8B870301 MOV AX,[0103+BX]

0124 3B870501 CMP AX,[0105+BX]

0128 7611 JNA 013B

012A 8B970501 MOV DX,[0105+BX]

012E 89870501 MOV [0105+BX],AX

0132 89870301 MOV [0103+BX],DX

DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.00= i.l.+...

DS:0010 1B 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FFA.

DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11A.

DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00J.

DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

- mov bx, 0
- mov byte [swap], 0
- Register values for **BX** and the memory location for **swap** (should be 0).
- **BX = 0** → compare data[0] & data[1]

DOSBox 0.74-3,...

AX	0000	SI	0000	CS	19F5	IP	0124	Stack	+0 0000	Flags	7200
BX	0000	DI	0000	DS	19F5				+2 20C0		
CX	0051	BP	0000	ES	19F5	HS	19F5		+4 9FFF	OF	DF
DX	0000	SP	FFFF	SS	19F5	FS	19F5		+6 E000	0	0
										1	0
										2	0
										3	0
										4	0
										5	0
										6	0
										7	0
										8	0
										9	0
										10	0
										11	0
										12	0
										13	0
										14	0
										15	0
										16	0
										17	0
										18	0
										19	0
										20	0
										21	0
										22	0
										23	0
										24	0
										25	0
										26	0
										27	0
										28	0
										29	0
										30	0
										31	0

0120 8B870301 MOV AX,[0103+BX]

0124 3B870501 CMP AX,[0105+BX]

0128 7611 JNA 013B

012A 8B970501 MOV DX,[0105+BX]

012E 89870501 MOV [0105+BX],AX

0132 89870301 MOV [0103+BX],DX

0136 C06170101 MOV [0117],01

013B 81C30200 ADD BX,0002

013F 81B12000 CMP BX,0012

DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.00= i.l.+...

DS:0010 1B 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FFA.

DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11A.

DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00J.

DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

The value from label is loaded into ax and now will now Be compared with the value on address 0x0105 with is 37(hex)

```

DOSBox 0.74-3,...
AX 000C SI 0000 CS 19F5 IP 012B Stack +0 0000 Flags 7204
BX 0000 DI 0000 DS 19F5
CX 0051 BP 0000 ES 19F5 HS 19F5
DX 0000 SP FF7E SS 19F5 FS 19F5

CMD >
0124 3B876501 CMP AX,[0165+BX]
012B 7611 JNA 013B
012A 8B76501 MOV DX,[0165+BX]
012E 8976501 MOV [0165+BX],DX
0132 8976301 MOV [0163+BX],DX
0136 C66170101 MOV [0117],01
013B 81C36200 ADD BX,0002
013F 81F81200 CMP BX,0012
0143 75DB JNZ 0120

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= 1.+.
DS:0010 10 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FF .....f. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF .....f. ....
DS:0030 A2 01 14 00 10 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

This command tells to jump if ax<=next number to 013B

Which means no swap will occur

```

DOSBox 0.74-3,...
AX 000C SI 0000 CS 19F5 IP 0136 Stack +0 0000 Flags 7204
BX 0000 DI 0000 DS 19F5
CX 0051 BP 0000 ES 19F5 HS 19F5
DX 0037 SP FF7E SS 19F5 FS 19F5

CMD >
0132 8976301 MOV [0163+BX],DX
0136 C66170101 MOV [0117],01
013B 81C36200 ADD BX,0002
013F 81F81200 CMP BX,0012
0143 75DB JNZ 0120
0145 803E170101 CMP [0117],01
014A 74CC JZ 0118
014C B0004C MOV AX,4C00
014F CD21 INT 21

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= 1.+.
DS:0010 10 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FF .....f. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF .....f. ....
DS:0030 A2 01 14 00 10 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Over here we could see that the swap has occurred

Through DX register and the value at memory address 0x0105 has been swapped with value at 0x0105

After this , as swap has occurred swap flag will turn to 1 as shown below:

```

DOSBox 0.74-3,...
AX 000C SI 0000 CS 19F5 IP 013B Stack +0 0000 Flags 7204
BX 0000 DI 0000 DS 19F5
CX 0051 BP 0000 ES 19F5 HS 19F5
DX 0037 SP FF7E SS 19F5 FS 19F5

CMD >
0136 C66170101 MOV [0117],01
013B 81C36200 ADD BX,0002
013F 81F81200 CMP BX,0012
0143 75DB JNZ 0120
0145 803E170101 CMP [0117],01
014A 74CC JZ 0118
014C B0004C MOV AX,4C00
014F CD21 INT 21
0151 7E08 JNE 015B

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= 1.+.
DS:0010 10 01 10 01 10 01 92 01 01 01 01 00 02 FF FF FF .....f. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF .....f. ....
DS:0030 A2 01 14 00 10 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Now as highlighted swap flag turned to 1

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 013F Stack +0 0000 Flags 7200
BX 0002 DI 0000 DS 19F5 +2 20C0
CX 0051 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0037 SP FFFE SS 19F5 FS 19F5 +6 E000 0 0 1 0 0 0 0 0

[CMD >]
013B 81C30200 ADD BX,0002
013F 81FB1200 CMP BX,0012
0143 75DB JNZ 0120
0145 803E170101 CMPEB [0117],01
0146 740C JZ 0118
014C B0004C MOV AX,4C00
014F CD21 INT 21
0151 7E08 JNG 015B
0153 B00100 MOV AX,0001

[Registers]
0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 1B 01 10 01 1B 01 92 01 01 01 01 00 02 FF FF FF .....A. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....f. ....
DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

[Flags]
ZF 0

```

Now bx has been converted to 2

- **BX = 2** → compare data[1] & data[2]

And then 2 is compared with 18 which tells weather whole

Array has been traversed or not

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0120 Stack +0 0000 Flags 7205
BX 0002 DI 0000 DS 19F5 +2 20C0
CX 0051 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0037 SP FFFE SS 19F5 FS 19F5 +6 E000 0 0 1 0 0 1 1 1

[CMD >]
0143 75DB JNZ 0120
0145 803E170101 CMPEB [0117],01
0146 740C JZ 0118
014C B0004C MOV AX,4C00
014F CD21 INT 21
0151 7E08 JNG 015B
0153 B00100 MOV AX,0001

[Registers]
0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 1B 01 10 01 1B 01 92 01 01 01 01 00 02 FF FF FF .....A. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....f. ....
DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

[Flags]
ZF 1

```

Here debugger checks if zero flag has been raised or not

If zf is raised jnz will not be executed

```

DOSBox 0.74-3,...
AX 0007 SI 0000 CS 19F5 IP 013F Stack +0 0000 Flags 7204
BX 0012 DI 0000 DS 19F5 +2 20C0
CX 0051 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0000 SP FFFE SS 19F5 FS 19F5 +6 E000 0 0 1 0 0 1 0 0

[CMD >]
013B 81C30200 ADD BX,0002
013F 81FB1200 CMP BX,0012
0143 75DB JNZ 0120
0145 803E170101 CMPEB [0117],01
0146 740C JZ 0118
014C B0004C MOV AX,4C00
014F CD21 INT 21
0151 7E08 JNG 015B
0153 B00100 MOV AX,0001

[Registers]
0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 1B 01 10 01 1B 01 92 01 01 01 01 00 02 FF FF FF .....A. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....f. ....
DS:0030 A2 01 14 00 1B 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

[Flags]
ZF 0

```

Now we notice:

All array has been traversed as bx contains 18(in hex 12)

Swap flag has changed to 1

Zero flag is 0 this indicating no jump happens,

confirming that sorting is complete.

Now we can see array has been sorted.

AX 0037	SI 0000	CS 19F5	IP 014C	Stack +0 0000	Flags 7295
BX 0012	DI 0000	DS 19F5		+2 20CD	
CX 0051	BP 0000	ES 19F5	HS 19F5	+4 9FFF	OF DF IF SF ZF AF PF CF
DX 0000	SP FFFE	SS 19F5	FS 19F5	+6 EA00	0 0 1 1 0 1 1 1
{R} reg=value					
CMD >SSS					
014A 74CC JZ 0118				DS:0100 E9 15 00 00 00 0A 00 19	
014C B8004C MOV AX,4C00				DS:0108 00 1E 00 23 00 28 00 2D	
014F CD21 INT 21				DS:0110 00 32 00 37 00 3C 00 00	
0151 0000 ADD [BX+SI],AL				DS:0118 BB 00 00 C6 06 17 01 00	
				DS:0120 8B 87 03 01 3B 87 05 01	

After complete sorting the program will moves towards the terminate statement and the program will end with the sorted array.